KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

함수 2
기본값과
변수 범위

자주 쓰는 값은 미리 채워 두고
함수 안과 밖의 변수 구분하기

```
def order(menu_name, size="M"):
order("latte")
order(size="L", menu_name="tea")
count = 0
def visit(count):
    return count + 1
```

오늘의 순서

1 왜 필요한가

매번 적는 M, 사라진 total, 기본값과 안팎 그림, 가장 짧은 기본값

4–6쪽

2 기본값과 키워드 인자

여러 기본값, 칸 이름으로 넣기, 순서 규칙, print와 sorted에서 본 키워드 인자

7–10쪽

3 지역 변수

안에서 만든 변수, 매개변수, 부를 때마다 새로

11–13쪽

4 전역 변수

밖의 변수 읽기, 같은 이름, UnboundLocalError, global, return과 비교

14–18쪽

5 리스트와 딕셔너리 넘기기

append로 원본 바뀌기, 숫자와 비교, 재고 줄이기, copy

19–22쪽

6 실습

주문 함수, 영수증 함수, 포인트 적립

23–25쪽

3쪽은 노트북 준비, 26–27쪽은 막힐 때와 한 장 요약이에요.

들어가기

19에서 이어서 해요

- ✓ hello-uv 노트북, 커널 .venv — 09 · 25–27쪽
- ✓ print의 sep, end와 copy() — 13
- ✓ 딕셔너리 get, stock[name] -= 1 — 15
- ✓ sorted(..., reverse=True) — 16
- ✓ if, for, while — 17, 18
- ✓ def, 매개변수, return, None — 19

기억이 안 나면 오른쪽에 적힌 챕터를 먼저 다시 봐요.

노트북 다시 열기

1. VS Code로 hello-uv 폴더 열기
2. 탐색기에서 `basics.ipynb` 열기
3. 오른쪽 위 커널이 .venv인지 보기
4. 셀에 적고 `Shift` + `Enter` 로 실행

1. 왜 필요한가

19의 함수로 쓰다 보면 두 군데서 막혀요

거의 늘 같은 값을 넣어요

[1]
```python
def order(menu_name, size):
    print(f"{menu_name} {size} 사이즈")

order("latte", "M")
order("coffee", "M")
order("tea", "M")
```

```
latte M 사이즈
coffee M 사이즈
tea M 사이즈
```

[2]
```python
order("cake")
```

```
TypeError: order() missing 1 required positional argument: 'size'
```

M을 매번 적어야 해요

빼면 TypeError예요. 안 적으면 M으로 해 주면 좋겠어요.

함수 안에서 만든 값이 안 보여요

[3]
```python
def calc():
    total = 4500 * 2

calc()
print(total)
```

```
NameError: name 'total' is not defined
```

total이 어디 갔나요

calc 안에서 분명히 만들었는데 밖에서는 없다고 나와요.

오늘은 기본값과 변수가 보이는 범위를 배워요.

1. 왜 필요한가

칸에 미리 채운 값, 그리고 안과 밖

기본값: 칸에 미리 넣어 둔 값

```
def order(menu_name, size="M"):
```

menu_name	size
꼭 넣어요	"M"

부를 때 빼면 "M"이 들어가요. 넣으면 넣은 값으로 바뀌어요.

밖: 노트북 전체 (전역)

`shop = "KANT 카페"` `count = 0`

안: def calc() 몸통 (지역)

`total = 9000`

`매개변수도 여기`

함수가 끝나면 이 상자는 사라져요

안에서 밖은 보여요

밖에서 안은 안 보여요

함수 안에서는 밖이 보이고, 밖에서는 안이 보이지 않아요.

1. 왜 필요한가

def 줄에서 칸 = 값으로 기본값을 정해요

[4]
```python
def order(menu_name, size="M"):          size="M"
    print(f"{menu_name} {size} 사이즈")

order("latte")
order("latte", "L")
```
Python

```
latte M 사이즈
latte L 사이즈
```

부르기	size	
`order("latte")`	`"M"`	기본값
`order("latte", "L")`	`"L"`	넣은 값

빼도 되는 칸이 생겼어요

기본값이 있는 칸은 부를 때 빼도 돼요. 기본값이 없는 menu_name은 지금처럼 꼭 넣어요.

= 앞뒤를 띄우지 않아요

변수에 넣을 땐 x = 1처럼 띄웠지만, 괄호 안 기본값은 size="M"처럼 붙여 쓰는 게 파이썬 관례예요.

기본값이 있는 칸은 빼면 기본값, 넣으면 넣은 값이에요.

2. 기본값과 키워드 인자

기본값이 여러 개면 앞에서부터 채워요

[1]
```python
def order(menu_name, size="M", qty=1):
    print(f"{menu_name} {size} {qty}잔")

order("latte")
order("latte", "L")
order("latte", "L", 2)
order("latte", qty=3)
```

```
latte M 1잔
latte L 1잔
latte L 2잔
latte M 3잔
```

넣은 값은 앞 칸부터

"L"은 둘째 칸 size에, 2는 셋째 칸 qty에 들어가요. 넣지 않은 칸만 기본값이에요.

부르기	size	qty
`order("latte")`	`"M"`	`1`
`order("latte", "L")`	`"L"`	`1`
`order("latte", "L", 2)`	`"L"`	`2`
`order("latte", qty=3)`	`"M"`	`3`

qty만 바꾸고 싶으면

순서대로 넣으면 size를 먼저 적어야 해요. qty=3처럼 칸 이름을 붙이면 size는 기본값으로 두고 qty만 바꿔요.

칸 이름 = 값으로 넣으면 원하는 칸만 고를 수 있어요.

2. 기본값과 키워드 인자

칸 이름을 붙이면 순서를 바꿔도 돼요

[2]
```python
order(size="L", menu_name="tea")
order("tea", qty=2, size="S")
```

```
tea L 1잔
tea S 2잔
```

`order(size="L", menu_name="tea")`

menu_name	size	qty
"tea"	"L"	1

"tea" 위치 인자. 적은 순서대로 칸에 들어가요

qty=2 키워드 인자. 칸 이름을 보고 들어가요

같이 쓸 땐 순서대로 먼저

order("tea", qty=2, size="S")처럼 위치 인자를 앞에, 키워드 인자를 뒤에 적어요.

키워드 인자는 칸 이름을 적어서 넣어요. 순서를 외우지 않아도 돼요.

2. 기본값과 키워드 인자

기본값과 키워드 인자는 뒤에 적어요

[3]
```python
def order(size="M", menu_name):
    print(menu_name, size)
```

```
SyntaxError: parameter without a default follows parameter with a default
```

[4]
```python
order(size="L", "tea")
```

```
SyntaxError: positional argument follows keyword argument
```

만들 때: 기본값 칸은 뒤로

기본값 없는 칸 뒤에 기본값 칸을 둬요. def order(menu_name, size="M"):

부를 때: 키워드 인자는 뒤로

이름 없는 값이 이름 붙인 값 뒤에 오면 안 돼요. order("tea", size="L")

두 오류 모두 실행 전에 SyntaxError로 알려 줘요.

2. 기본값과 키워드 인자

키워드 인자는 이미 써 봤어요

[5]
```python
print("coffee", "latte", sep=" / ")
print("coffee", end=" ")
print("latte")
print(sorted([5000, 4500, 6500], reverse=True))
```

```
coffee / latte
coffee latte
[6500, 5000, 4500]
```

print()만 쳐도 줄이 바뀌는 이유

end의 기본값이 줄바꿈이라서 print()는 빈 줄 하나를 찍어요. 18의 구구단에서 쓴 그 print()예요.

`sep=" / "` print 13

값 사이에 넣을 글자. 기본값은 빈칸 하나

`end=" "` print 13

끝에 붙일 글자. 기본값은 줄바꿈

`reverse=True` sorted 16

큰 것부터. 기본값은 False

파이썬이 만든 함수도 기본값과 키워드 인자로 만들어져 있어요.

3. 지역 변수

함수 안에서 만든 변수는 밖에서 안 보여요

[1]
```python
def calc():
    total = 4500 * 2
    print("안:", total)

calc()
print("밖:", total)
```
여기서 오류 Python

```
안: 9000

NameError: name 'total' is not defined
```

지역 변수

함수 안에서 만든 변수예요. 그 함수 안에서만 쓰고, 함수가 끝나면 사라져요. 밖에서 쓰려면 19처럼 return으로 돌려받아요.

calc()를 부르는 동안

`total = 9000`

몸통 안에서는 total을 쓸 수 있어요

calc()가 끝난 뒤

~~`total = 9000`~~

상자가 사라져서 밖에서는 찾을 수 없어요

함수 안의 값을 밖에서 쓰려면 return으로 돌려받아요.

3. 지역 변수

매개변수도 함수 안에서만 쓰는 변수예요

[2]
```python
def greet(name):
    print(f"{name}님, 어서 오세요")

greet("kim")
print(name)  # 밖
```

```
kim님, 어서 오세요


NameError: name 'name' is not defined
```

그래서 이름이 겹쳐도 괜찮아요

19의 calc_total(cart, menu)에서 menu 칸은 바깥 menu 딕셔너리와 이름만 같은 다른 칸이에요.

greet("kim") 안

`name = "kim"`

부를 때 넣은 값이 담긴 칸도 함수 안에만 있어요.

밖에서 쓰려면

밖에서 만든 변수를 인자로 넣거나, 함수가 return한 값을 받아요.

3. 지역 변수

부를 때마다 처음부터 새로 만들어요

[3]
```python
def add_stamp():
    stamp = 0
    stamp += 1
    print("도장", stamp)

add_stamp()
add_stamp()
add_stamp()
```

```
도장 1
도장 1
도장 1
```

첫째 부르기

`stamp = 0`

`stamp += 1`

1

둘째 부르기

`stamp = 0`

`stamp += 1`

1

셋째 부르기

`stamp = 0`

`stamp += 1`

1

쌓으려면 바깥에 둬야 해요

다음 파트에서 바깥 변수를 읽고 바꾸는 법을 봐요.

⚠ 도장이 쌓이지 않아요

부를 때마다 새 상자에서 stamp = 0부터 시작해요. 끝나면 상자째 사라져서 지난번 값을 기억하지 못해요.

지역 변수는 부를 때마다 새로 생기고, 끝나면 사라져요.

4. 전역 변수

함수 밖에서 만든 변수는 안에서 읽을 수 있어요

[1]
```python
shop = "KANT 카페"                    # 밖에서 만듦

def hello():
    print(f"{shop}에 어서 오세요")    # 안에서 읽음

hello()
```

```
KANT 카페에 어서 오세요
```

전역 변수

함수 밖, 노트북 셀에서 만든 변수예요. 모든 함수 안에서 읽을 수 있어요. 가게 이름처럼 바뀌지 않는 값에 써요.



함수 안에서 이름을 찾을 땐 안부터 보고, 없으면 밖을 봐요.

4. 전역 변수

안에서 같은 이름에 넣으면 새 지역 변수가 생겨요

[2]
```python
price = 4500

def set_price():
    price = 5000                    # 새 지역 변수
    print("안:", price)

set_price()
print("밖:", price)
```

```
안: 5000
밖: 4500
```

밖 price

4500

안 price

5000

가격을 바꾸려던 거라면

바깥 price는 그대로 4500이에요. 바깥 값을 바꾸는 법은 두 장 뒤에 봐요.

이름만 같은 다른 상자예요

함수 안에서 price = 5000처럼 값을 넣으면 안쪽에 price를 새로 만들어요. 바깥 price는 손대지 않아요. 함수가 끝나면 안쪽 price만 사라져요.

함수 안에서 = 로 값을 넣으면 그 이름은 지역 변수가 돼요.

4. 전역 변수

읽다가 바꾸면 UnboundLocalError가 나요

[3]

```
count = 0

def visit():
    count = count + 1

visit()
```

count는 지역 변수로 정해짐

Python

```
UnboundLocalError: cannot access local variable 'count' where it is not associated with a value
```

1 **count = ... 이 함수 안에 있어요** — 그래서 count는 이 함수의 지역 변수예요

2 **오른쪽 count + 1을 먼저 계산해요** — 지역 count를 읽으려는데

3 **지역 count에는 아직 값이 없어요** — UnboundLocalError

앞 장과 같은 규칙이에요

함수 안에서 = 로 넣는 이름은 지역 변수예요. 읽기만 하면 바깥을 보지만, 넣는 순간 지역 변수가 돼요.

4. 전역 변수

global을 적으면 바깥 변수를 바꿔요

[4]
```python
count = 0

def visit():
    global count                    바깥 count를 써요
    count = count + 1

visit()
visit()
visit()
print(count)
```

```
3
```

밖 count

0 → 3

global count

함수 첫 줄에 적으면, 이 함수 안의 count는 바깥 count라는 뜻이에요.

⚠ 있다는 것만 알아 둬요

여러 함수가 바깥 값을 바꾸면 어디서 바뀌었는지 찾기 어려워요.

global은 꼭 필요할 때만 써요. 다음 장에서 더 나은 방법을 봐요.

4. 전역 변수

바깥 값을 바꿀 땐 return으로 돌려받아요

global로 바꾸기

[4]
```python
count = 0

def visit():
    global count
    count = count + 1

visit()
visit()
visit()
print(count)
```

```
3
```

return으로 돌려받기

[5]
```python
def visit(count):
    return count + 1

count = 0
count = visit(count)
count = visit(count)
count = visit(count)
print(count)
```

바뀌는 곳이 보여요

```
3
```

return 쪽을 권해요

count가 바뀌는 줄이 count = visit(count)로 눈에 보여요. 함수는 받은 값으로 계산해서 돌려주기만 해요.

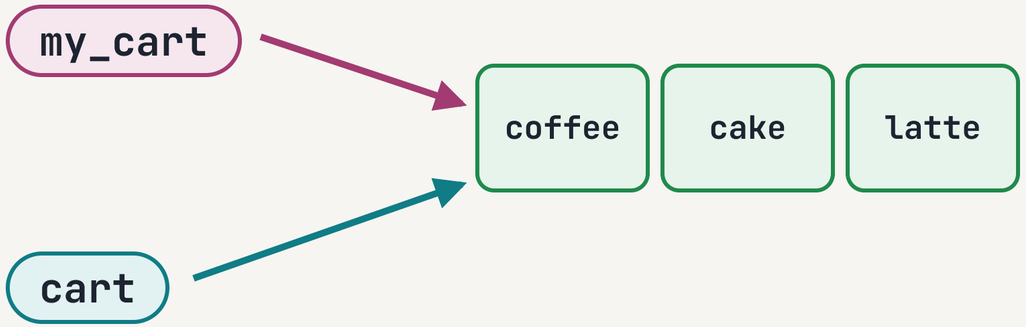
5. 리스트와 딕셔너리 넘기기

리스트를 넘기고 안에서 append하면 원본이 바뀌어요

[1]
```python
def add_item(cart, name):
    cart.append(name)

my_cart = ["coffee"]
add_item(my_cart, "cake")
add_item(my_cart, "latte")
print(my_cart)
```

```
['coffee', 'cake', 'latte']
```



return 없이도 바뀌었어요

함수가 새 값을 만든 게 아니라 받은 리스트 자체를 고쳤어요. 원본을 지키는 법은 22쪽에서 봐요.

이름 두 개, 리스트 하나

add_item(my_cart, ...)로 부르면 cart도 같은 리스트를 가리켜요. cart로 append하면 my_cart로 봐도 늘어나 있어요.

리스트를 넘기면 복사본이 아니라 같은 리스트가 들어가요.

5. 리스트와 딕셔너리 넘기기

숫자는 안에서 바꿔도 밖이 그대로예요

[2]
```python
def add_one(n):
    n += 1
    print("안:", n)

x = 1
add_one(x)
print("밖:", x)
```

```
안: 2
밖: 1
```

밖 x

1

안 n

1 → 2

`n += 1, n = ...`	새 값을 넣어요	**밖은 그대로**
`cart.append(...)`	받은 리스트를 고쳐요	**밖도 바뀌어요**
`stock[name] -= 1`	받은 딕셔너리 칸을 고쳐요	**밖도 바뀌어요**

5. 리스트와 딕셔너리 넘기기

딕셔너리를 넘기면 재고를 바로 줄일 수 있어요

[3]
```python
def sell(stock, name):
    stock[name] -= 1

stock = {"coffee": 3, "latte": 1}
sell(stock, "latte")
sell(stock, "coffee")
print(stock)
```

```
{'coffee': 2, 'latte': 0}
```

부르기 전

coffee	latte
3	1

두 번 부른 뒤

coffee	latte
2	0

global이 필요 없어요

stock = 새 딕셔너리로 바꾼 게 아니라 받은 딕셔너리의 칸만 고쳤어요. 그래서 바깥 stock이 그대로 바뀌어요.

받은 딕셔너리의 칸을 고치면 원본 딕셔너리가 바뀌어요.

5. 리스트와 딕셔너리 넘기기

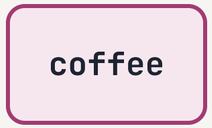
원본을 지키려면 copy()로 새로 만들어 돌려줘요

[4]
```python
def with_item(cart, name):
    new_cart = cart.copy()          # 복사본
    new_cart.append(name)
    return new_cart

my_cart = ["coffee"]
big_cart = with_item(my_cart, "cake")
print(my_cart)
print(big_cart)
```

```
['coffee']
['coffee', 'cake']
```

my_cart



big_cart



언제 어느 쪽?

원본을 고치는 게 목적이면 앞 장들처럼 바로 고쳐요. 원본은 두고 결과만 받고 싶으면 copy()로 새로 만들어 return해요.

함수가 받은 리스트를 고칠지, 새로 만들어 돌려줄지 정해서 써요.

6. 실습

주문 함수: 기본값과 재고 딕셔너리

[1]

```python
def take_order(stock, name, qty=1):
    if stock[name] < qty:
        print(f"{name} {qty}개: 재고가 {stock[name]}개뿐이에요")
        return False
    stock[name] -= qty
    print(f"{name} {qty}개 나갔어요")
    return True

stock = {"coffee": 3, "latte": 1, "cake": 2}
take_order(stock, "coffee", 2)
take_order(stock, "latte")
take_order(stock, "latte")
take_order(stock, "cake", qty=3)
print(stock)
```

Python

```
coffee 2개 나갔어요
latte 1개 나갔어요
latte 1개: 재고가 0개뿐이에요
cake 3개: 재고가 2개뿐이에요
{'coffee': 1, 'latte': 0, 'cake': 2}
```

qty=1

기본값 qty=1

한 개 주문이 대부분이라 빼고 불러요.

모자라면 끝

if 안의 return

재고가 모자라면 바로 나가요. 19에서 본 return이에요.

원본 재고

밖의 stock이 줄었어요

받은 딕셔너리 칸을 고쳤으니까요.

6. 실습

영수증 함수: 함수 안에서 함수 부르기

[2]
```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500}

def calc_total(cart, menu):
    total = 0
    for name, qty in cart.items():
        total += menu[name] * qty
    return total

def print_receipt(cart, menu, rate=0):      rate=0
    total = calc_total(cart, menu)          다른 함수
    sale = total * rate // 100
    for name, qty in cart.items():
        print(f"{name:<7}{qty}개 {menu[name] * qty:>7,}원")
    print(f"할인   {rate:>2}% {sale:>7,}원")
    print(f"결제       {total - sale:>7,}원")
```
Python

[3]
```python
cart1 = {"coffee": 2, "cake": 1}
cart2 = {"latte": 3, "cake": 2}
print_receipt(cart1, menu)
print("---")
print_receipt(cart2, menu, rate=10)
```
Python

```
coffee 2개    9,000원
cake   1개    6,500원
할인     0%        0원
결제          15,500원
---
latte  3개   15,000원
cake   2개   13,000원
할인    10%   2,800원
결제          25,200원
```

할인이 없으면 rate를 빼고, 있을 때만 rate=10처럼 이름을 붙여 넣어요.

6. 실습

적립 함수: get과 기본값으로 포인트 쌓기

[4]
```python
def add_point(points, name, amount, rate=5):
    points[name] = points.get(name, 0) + amount * rate // 100

points = {"kim": 300}
add_point(points, "kim", 9000)
add_point(points, "lee", 6500)
add_point(points, "lee", 10000, rate=10)
print(points)
```

Python

```
{'kim': 750, 'lee': 1325}
```

부르기	더한 값	합
kim 9000	450	750
lee 6500	325	325
lee 10000	1000	1325

rate=5

평소 적립률은 5%예요. 행사 때만 rate=10으로 넣어요.

get(name, 0)

처음 온 손님 lee는 키가 없어서 15의 기본값 0에서 시작해요.

return 없이 쌓여요

받은 딕셔너리에 키를 넣고 고쳤어요.

정리

막히면 오류의 마지막 줄을 봐요

1	`SyntaxError: parameter without a default follows ...` def 줄에서 기본값 칸을 앞에 뒀어요	기본값 칸을 뒤로
2	`SyntaxError: positional argument follows keyword argument` 부를 때 이름 없는 값을 뒤에 적었어요	위치 인자를 앞으로
3	`NameError: name 'total' is not defined` 함수 안의 지역 변수를 밖에서 썼어요	return으로 돌려받기
4	`UnboundLocalError: cannot access local variable` 함수 안에서 바깥 변수를 읽다가 바꿨어요	인자로 받고 return
5	**함수 안에서 바꿨는데 밖이 그대로예요** 안에서 = 로 새 지역 변수를 만들었어요	x = 함수(x)로 받기
6	**함수에 넘긴 리스트가 바뀌었어요** 받은 리스트를 append로 고쳤어요	copy()로 복사해 쓰기
7	**부를 때마다 값이 처음으로 돌아가요** 지역 변수는 부를 때마다 새로 생겨요	값을 밖에 두고 넘기기

헷갈리면 함수 안과 밖에서 print로 같은 변수를 찍어 비교해 봐요.

한 장 요약

`def f(a, b="M"):`

기본값은 뒤 칸에

`f(1, b=2)`

키워드 인자는 뒤에

지역 변수

안에서 만들고 끝나면 사라져요

전역 변수

밖에서 만들고 안에서 읽어요

`global x`

바깥 x 바꾸기, 되도록 피하기

`x = f(x)`

return으로 돌려받기

`cart.append()`

넘긴 리스트는 원본이 바뀌어요

`cart.copy()`

원본 지키기

함수 안에서 = 로 넣은 이름은 지역 변수

바깥 값을 바꿀 땐 global보다 인자와 return